Function_Write_Ups.cpp

```cpp
#include <iostream>

// ============================================================================
// EXERCISE 1: Singly Linked List - Remove Duplicates (Nested Pointers)
// Prompt: Write a function 'removeDuplicates' that modifies a singly linked
// list in-place so that only the first occurrence of each value is kept.
// Do not use standard containers or extra data structures.
// ============================================================================

struct Node
{
    int val;
    Node *next;
};

void removeDuplicates(Node *&head)
{
    if (head == nullptr || head->next == nullptr)
        return;

    Node *outer = head;
    while (outer != nullptr)
    {
        Node *inner = outer;
        while (inner->next != nullptr)
        {
            if (inner->next->val == outer->val)
            {
                Node *temp = inner->next;
                inner->next = inner->next->next;
                delete temp;
            }
            else
            {
                inner = inner->next;
            }
        }
        outer = outer->next;
    }
}

// ============================================================================
// EXERCISE 2: Array-Based List - Compact and Remove Item
// Prompt: Write a function 'deleteItemAndCompact' that searches an array
// for all occurrences of 'target', removes them by shifting remaining elements
// in-place, and updates 'length'.
// ============================================================================
```

```
48
49 void deleteItemAndCompact(int arr[], int &length, int target)
50 {
51     int write = 0;
52     for (int read = 0; read < length; read++)
53     {
54         if (arr[read] != target)
55         {
56             arr[write] = arr[read];
57             write++;
58         }
59     }
60     length = write;
61 }
62
63 // ============================================================================
64 // EXERCISE 3: Recursive Linked List - Remove Odd Nodes
65 // Prompt: Write a RECURSIVE function 'countAndRemoveOdd' that unlinks and
66 // deletes all nodes containing odd integer values from a linked list and
67 // returns the total count of deleted nodes.
68 // ============================================================================
69
70 int countAndRemoveOdd(Node *&head)
71 {
72     if (head == nullptr)
73         return 0;
74
75     int countFromRest = countAndRemoveOdd(head->next);
76
77     if (head->val % 2 != 0)
78     {
79         Node *temp = head;
80         head = head->next; // Updates original pointer via reference
81         delete temp;
82         return 1 + countFromRest;
83     }
84
85     return countFromRest;
86 }
87
88 // ============================================================================
89 // EXERCISE 4: Singly Linked List - Reverse In-Place (Iterative)
90 // Prompt: Write a function 'reverseList' that reverses a singly linked list
91 // in-place using three pointers (prev, curr, next).
92 // ============================================================================
93
94 void reverseList(Node *&head)
95 {
96     Node *prev = nullptr;
```

```
97      Node *curr = head;
98      Node *nextNode = nullptr;
99
100     while (curr != nullptr)
101     {
102         nextNode = curr->next; // Save next node
103         curr->next = prev;     // Reverse pointer
104         prev = curr;           // Advance prev
105         curr = nextNode;       // Advance curr
106     }
107
108     head = prev; // Update head pointer
109 }
110
111 // ============================================================================
112 // EXERCISE 5: Array-Based Queue - Insert at Position (Shift Right)
113 // Prompt: Write a function 'insertAt' that inserts 'value' at index 'pos' in
114 // an array-based list, shifting elements to the right. Increment 'length'.
115 // If 'pos' is out of bounds (pos < 0 or pos > length), return false.
116 // ============================================================================
117
118 bool insertAt(int arr[], int &length, int capacity, int pos, int value)
119 {
120     if (pos < 0 || pos > length || length >= capacity)
121     {
122         return false;
123     }
124
125     // Shift elements right starting from the end
126     for (int i = length; i > pos; i--)
127     {
128         arr[i] = arr[i - 1];
129     }
130
131     arr[pos] = value;
132     length++;
133     return true;
134 }
135
136 // ============================================================================
137 // MAIN FUNCTION (Demonstration & Output Verification)
138 // ============================================================================
139
140 void printList(Node *head)
141 {
142     while (head != nullptr)
143     {
144         std::cout << "[" << head->val << "] -> ";
145         head = head->next;
```

```cpp
    }
    std::cout << "nullptr\n";
}

int main()
{
    std::cout << "--- 1. Remove Duplicates ---\n";
    Node *list1 = new Node{3, new Node{1, new Node{3, new Node{2, new Node{1, nullptr}}}}};
    removeDuplicates(list1);
    printList(list1);

    std::cout << "\n--- 2. Delete and Compact ---\n";
    int arr2[] = {4, 1, 4, 4, 3, 2};
    int len2 = 6;
    deleteItemAndCompact(arr2, len2, 4);
    for (int i = 0; i < len2; i++)
        std::cout << arr2[i] << " ";
    std::cout << "\n";

    std::cout << "\n--- 3. Count & Remove Odd (Recursive) ---\n";
    Node *list3 = new Node{4, new Node{7, new Node{1, new Node{8, new Node{3, nullptr}}}}};
    int removed = countAndRemoveOdd(list3);
    std::cout << "Removed: " << removed << " nodes\nRemaining: ";
    printList(list3);

    std::cout << "\n--- 4. Reverse List ---\n";
    Node *list4 = new Node{10, new Node{20, new Node{30, nullptr}}};
    reverseList(list4);
    printList(list4);

    std::cout << "\n--- 5. Insert At Position ---\n";
    int arr5[10] = {10, 20, 40, 50};
    int len5 = 4;
    insertAt(arr5, len5, 10, 2, 30); // Insert 30 at index 2
    for (int i = 0; i < len5; i++)
        std::cout << arr5[i] << " ";
    std::cout << "\n";

    return 0;
}
```